# Introduction to Kserve

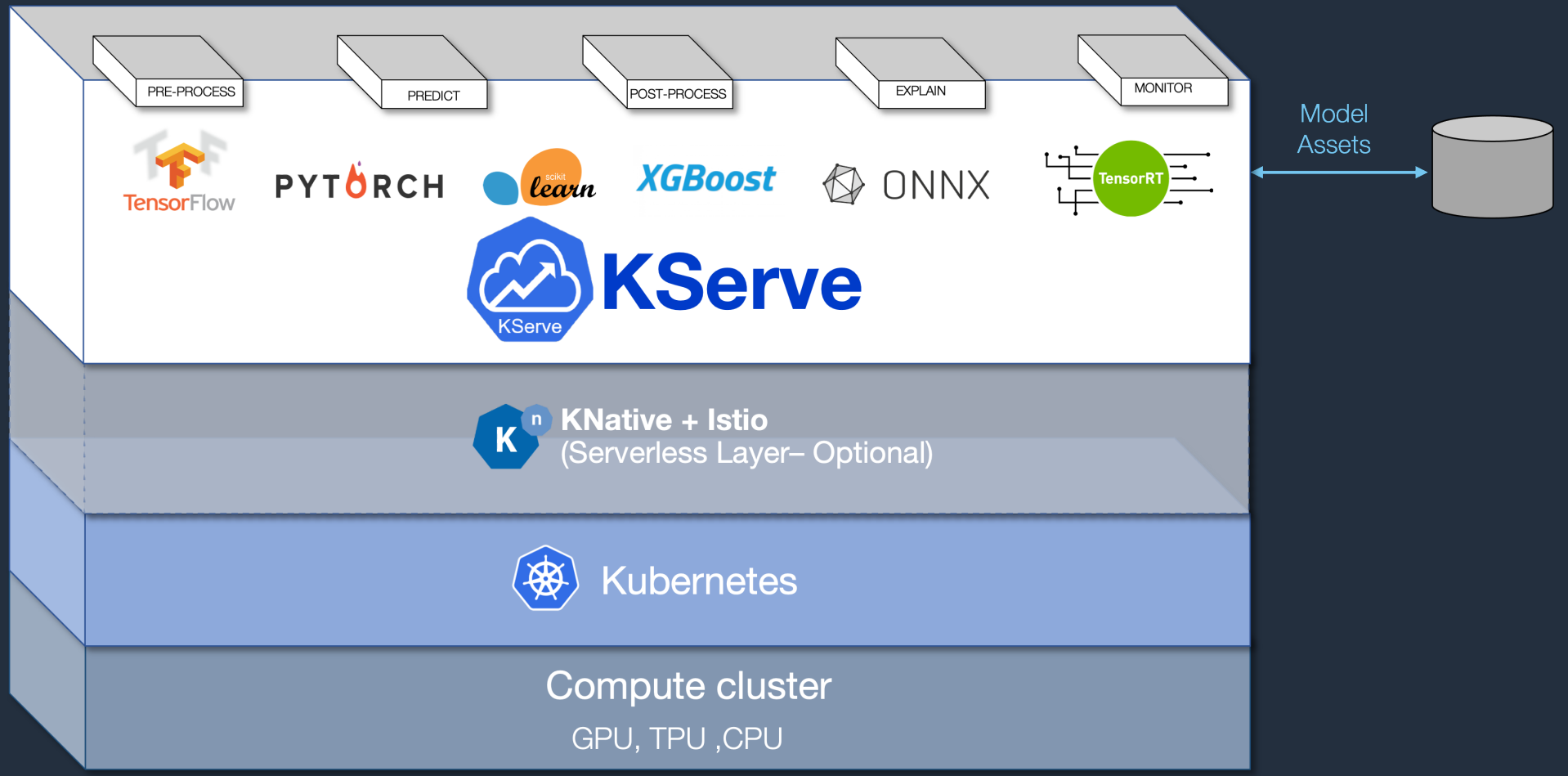
Keita Watanabe

Sr. Solutions Architect, AI/ML Frameworsk
AWS

# Agenda

- KServe Overview
- KServe Components
  - Inference Service
  - Predictor
  - AutoScaling with Knative Pod Autoscaler (KPA)
- ML inference with KServe Examples

# KServe



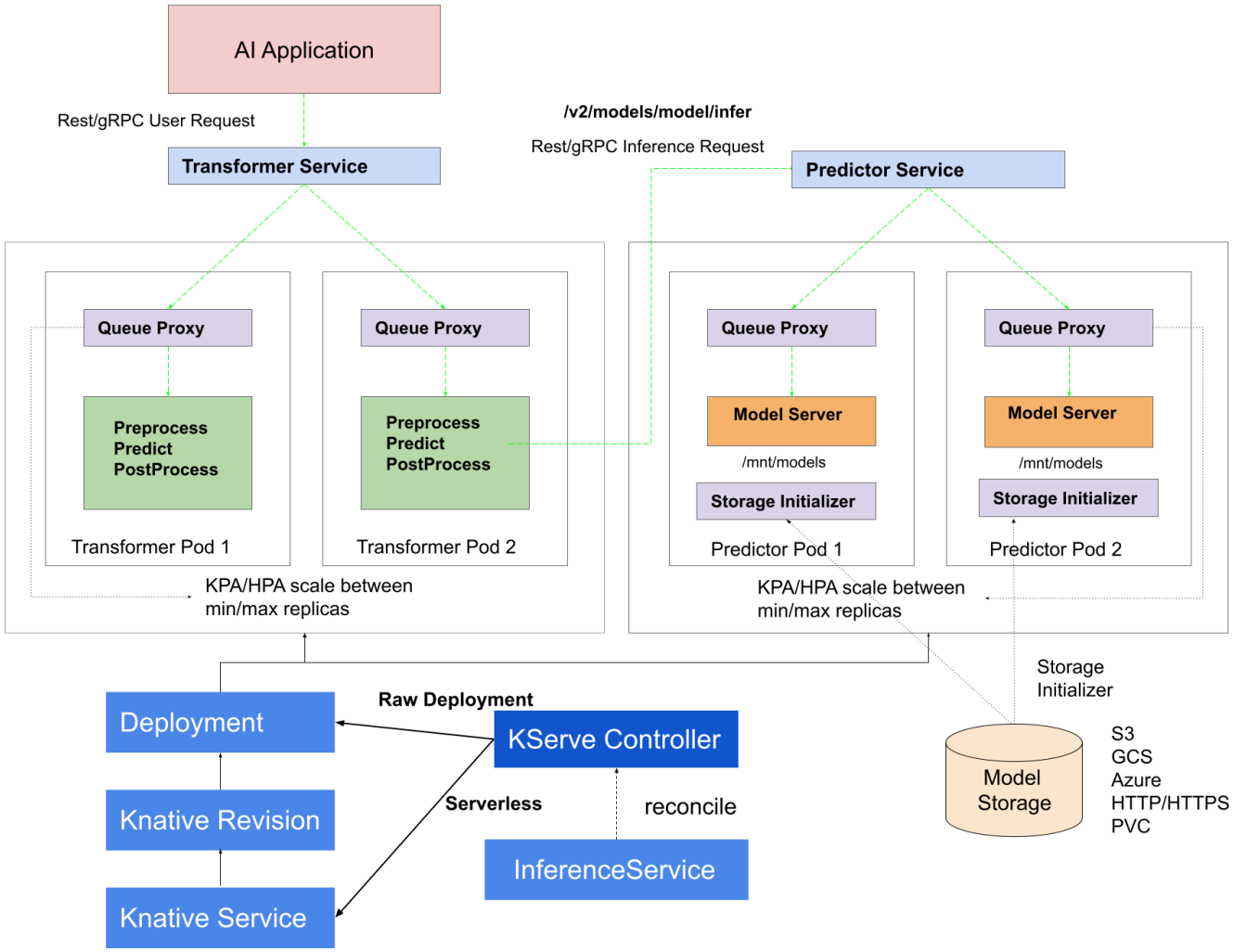



https://kserve.github.io/website/master/

# KServe Features

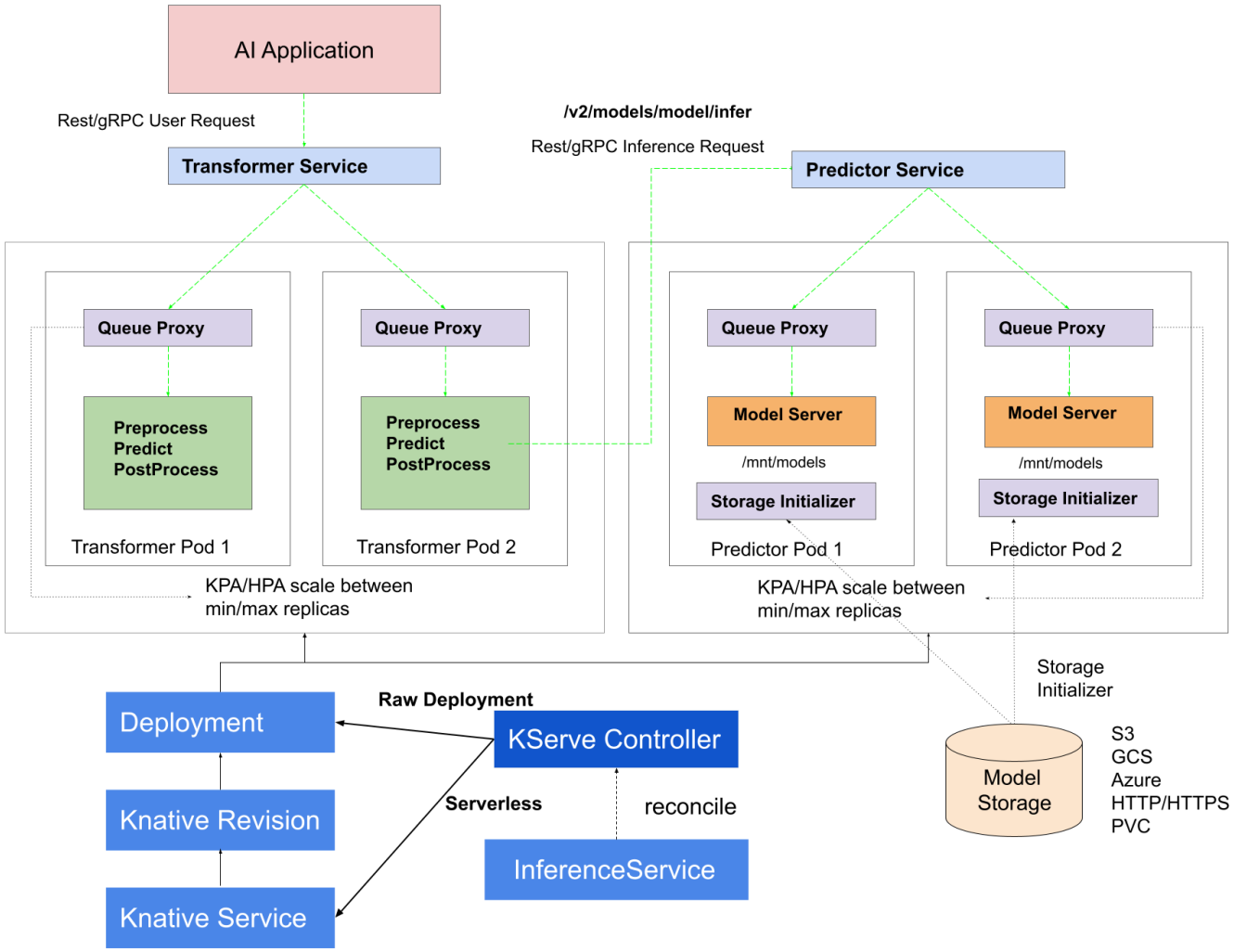
- Scale to and from Zero
- Request based Autoscaling
- Batching
- Request/Response logging
- Traffic management
- Security with AuthN/AuthZ
- Distributed Tracing
- Out-of-the-box metrics

# KServe Control Plane



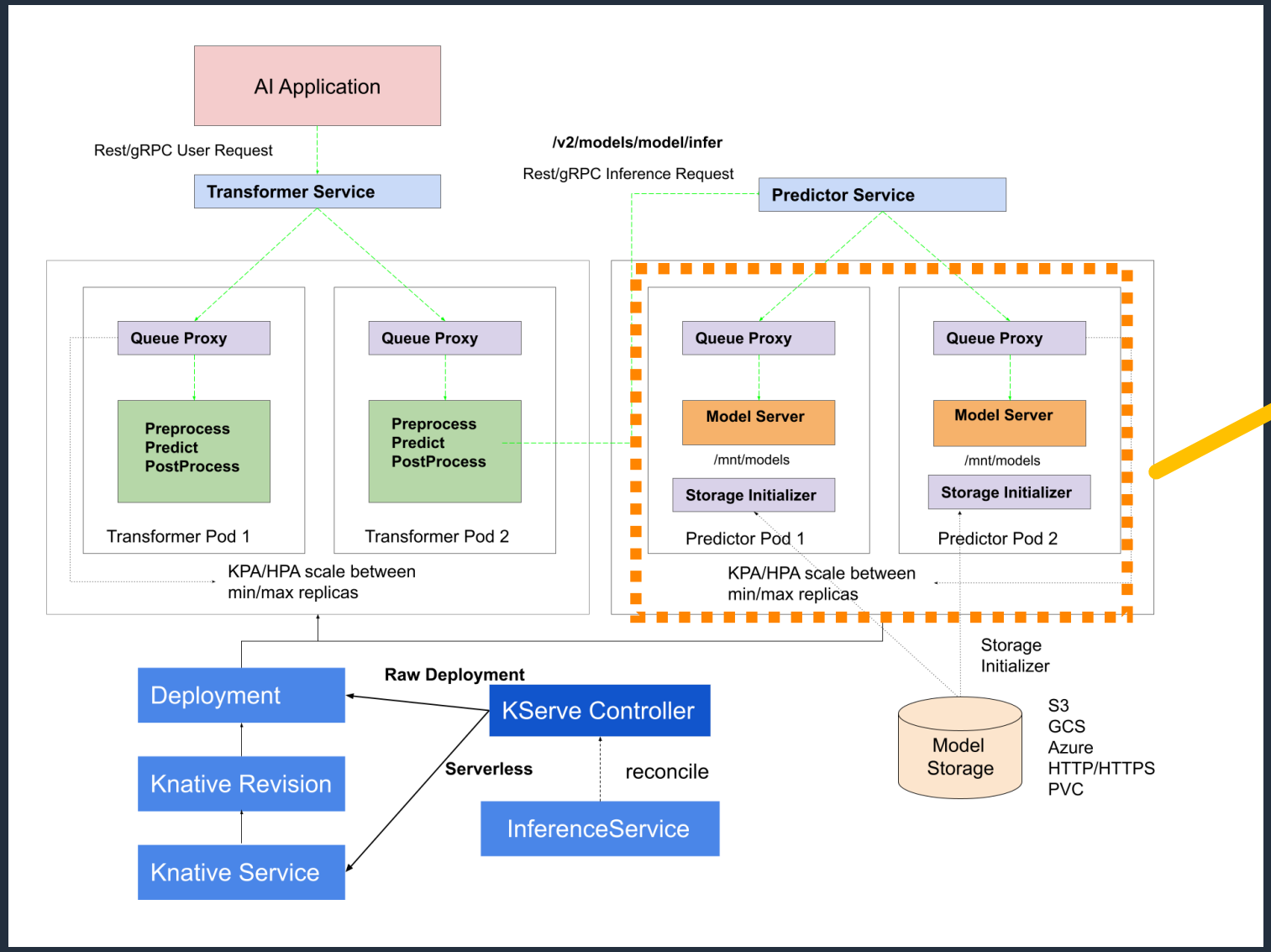

- Responsible for reconciling the InferenceService custom resources.
- It creates the Knative serverless deployment for predictor, transformer to enable autoscaling based on incoming request workload including scaling down to zero when no traffic is received.

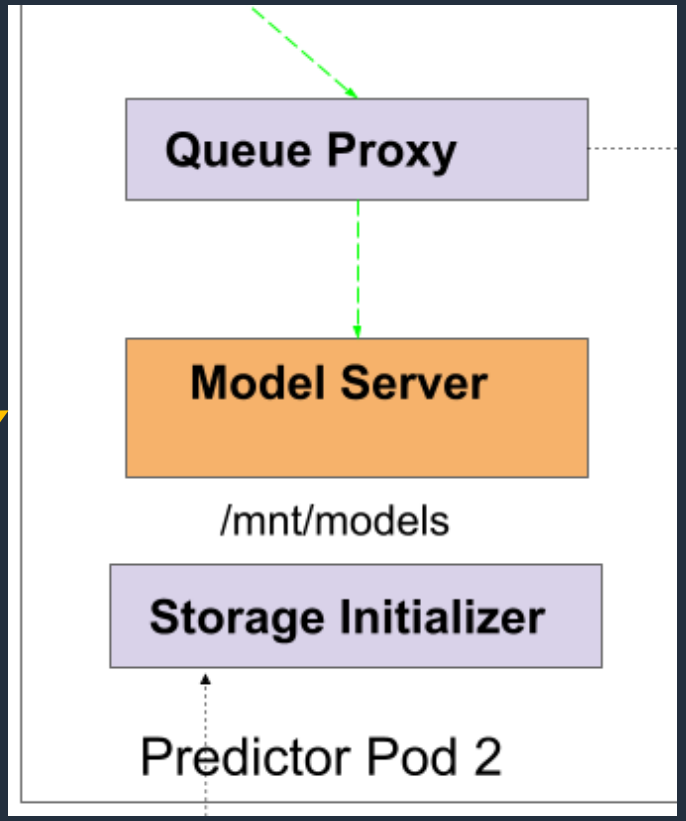
# KServe Control Plane




- Responsible for reconciling the InferenceService custom resources.
- It creates the Knative serverless deployment for predictor, transformer to enable autoscaling based on incoming request workload including scaling down to zero when no traffic is received.

# Predictor

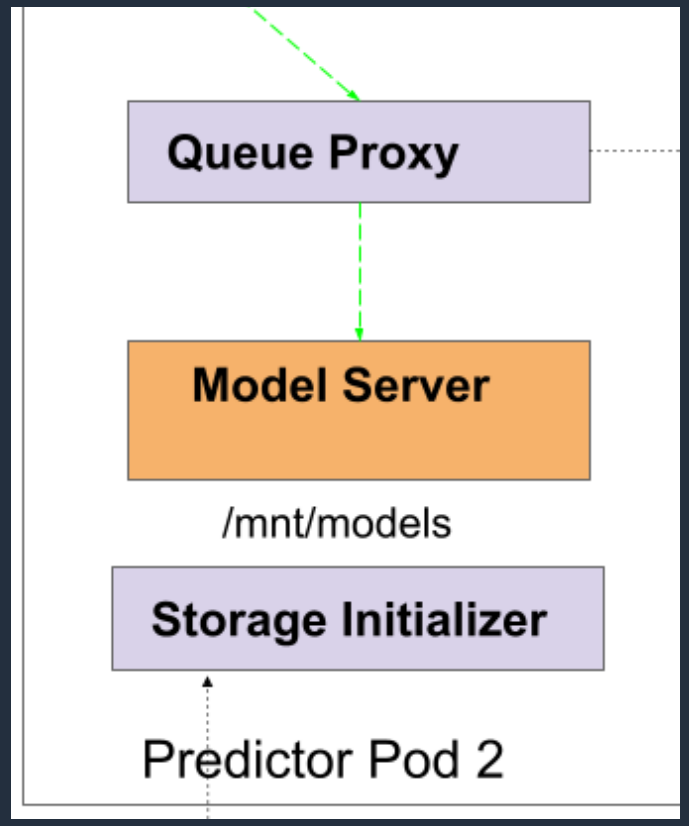





https://kserve.github.io/website/master/

# Predictor




Queue Proxy measures and limit concurrency to the user's application

Model Server deploys, manages, and serves machine learning models

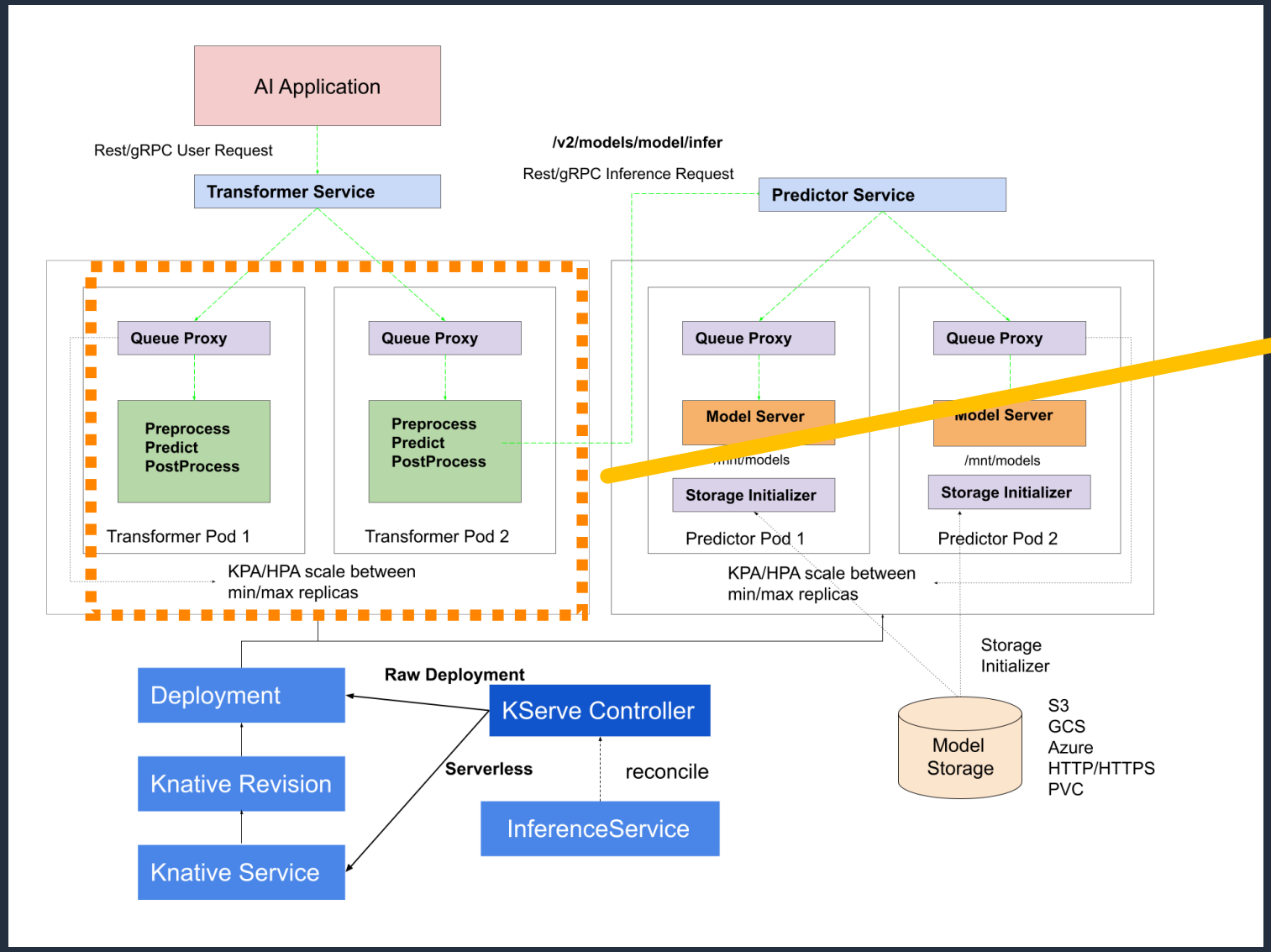
Storage Initializer retrieves and prepares machine learning models from various storage backends like Amazon S3

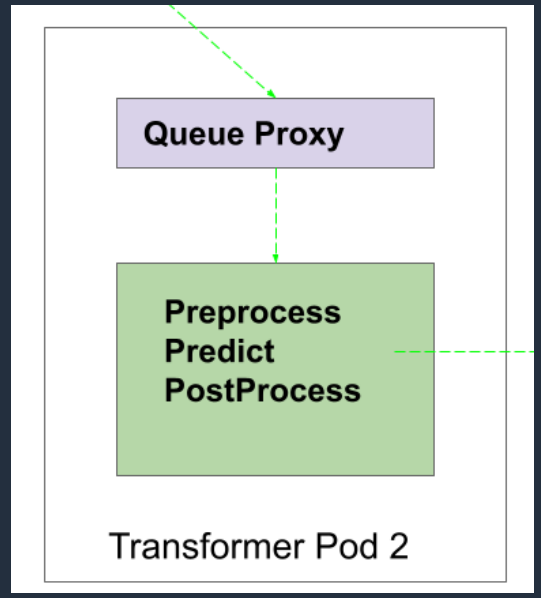




https://kserve.github.io/website/master/

# Transformer

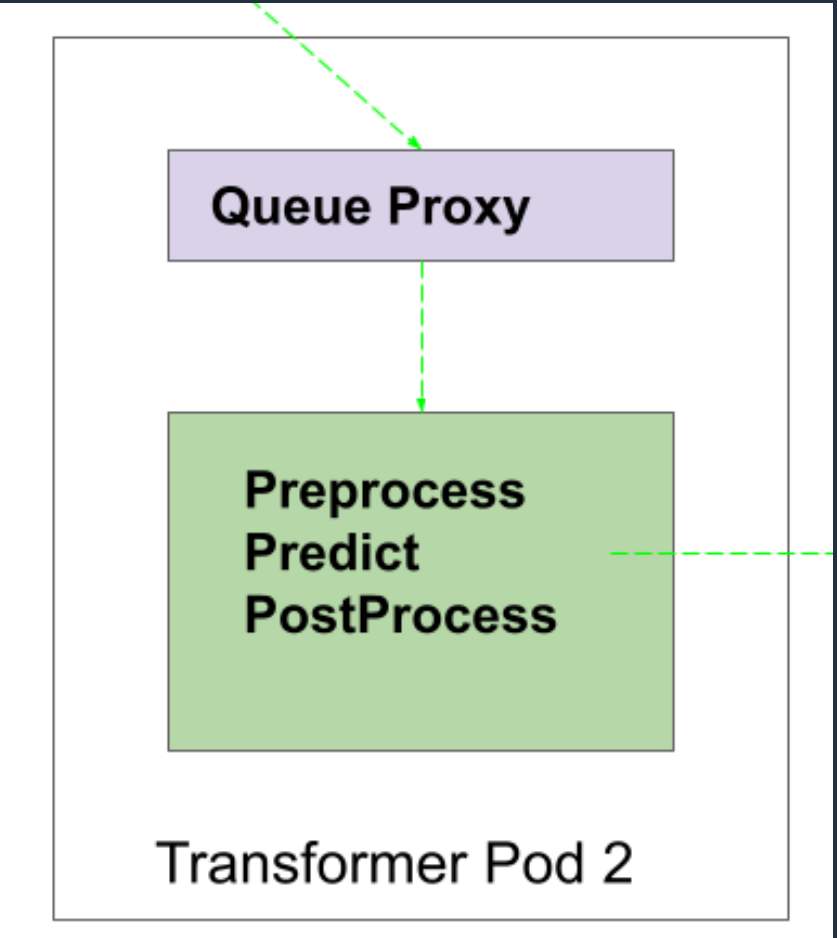





https://kserve.github.io/website/master/

# Transformer




Queue Proxy measures and limits concurrency to the user's application.

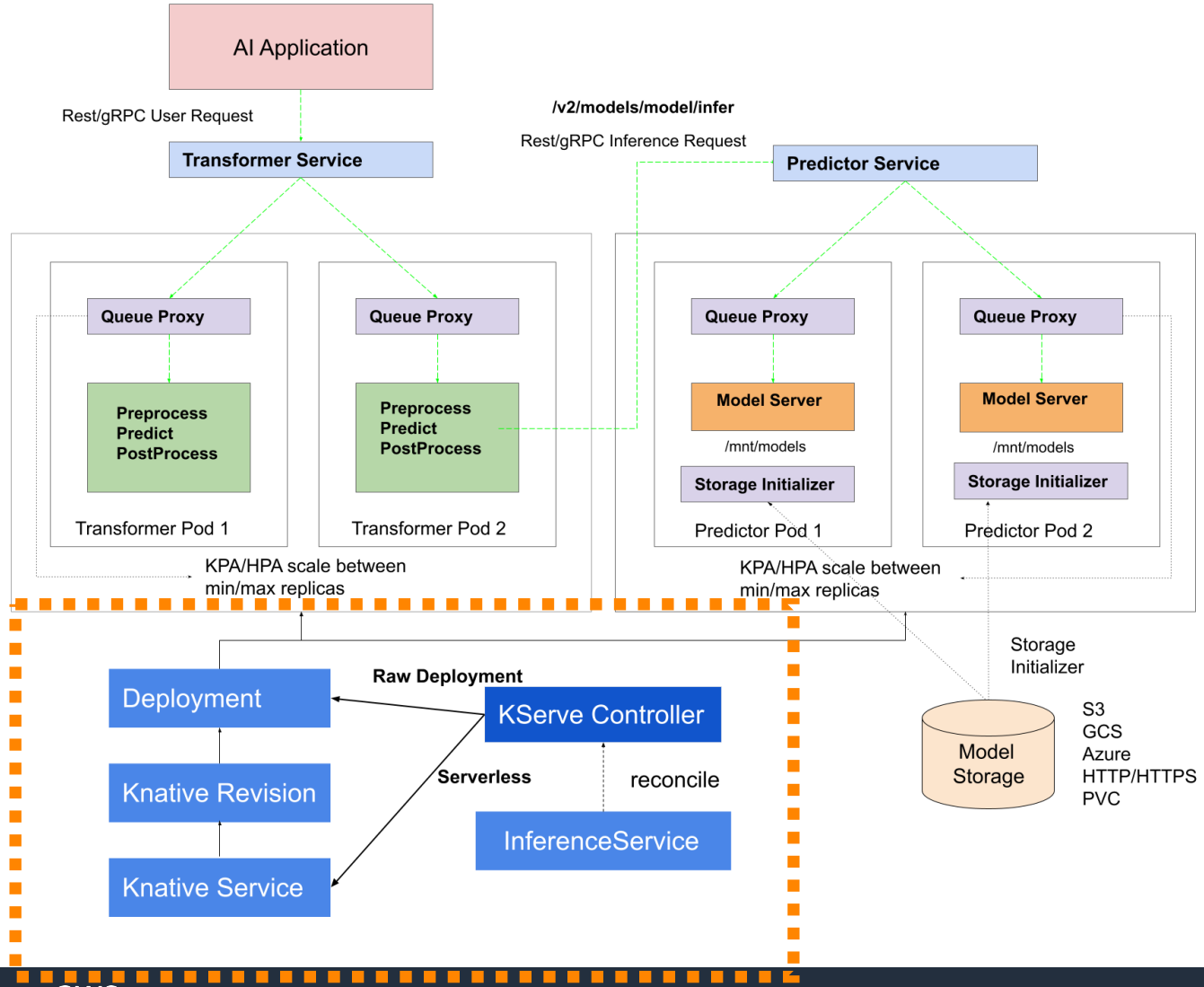
Model Server preprocesses input data and postprocesses output predictions, enabling seamless integration of custom logic or data transformations with the deployed machine learning models for improved model serving and inference.

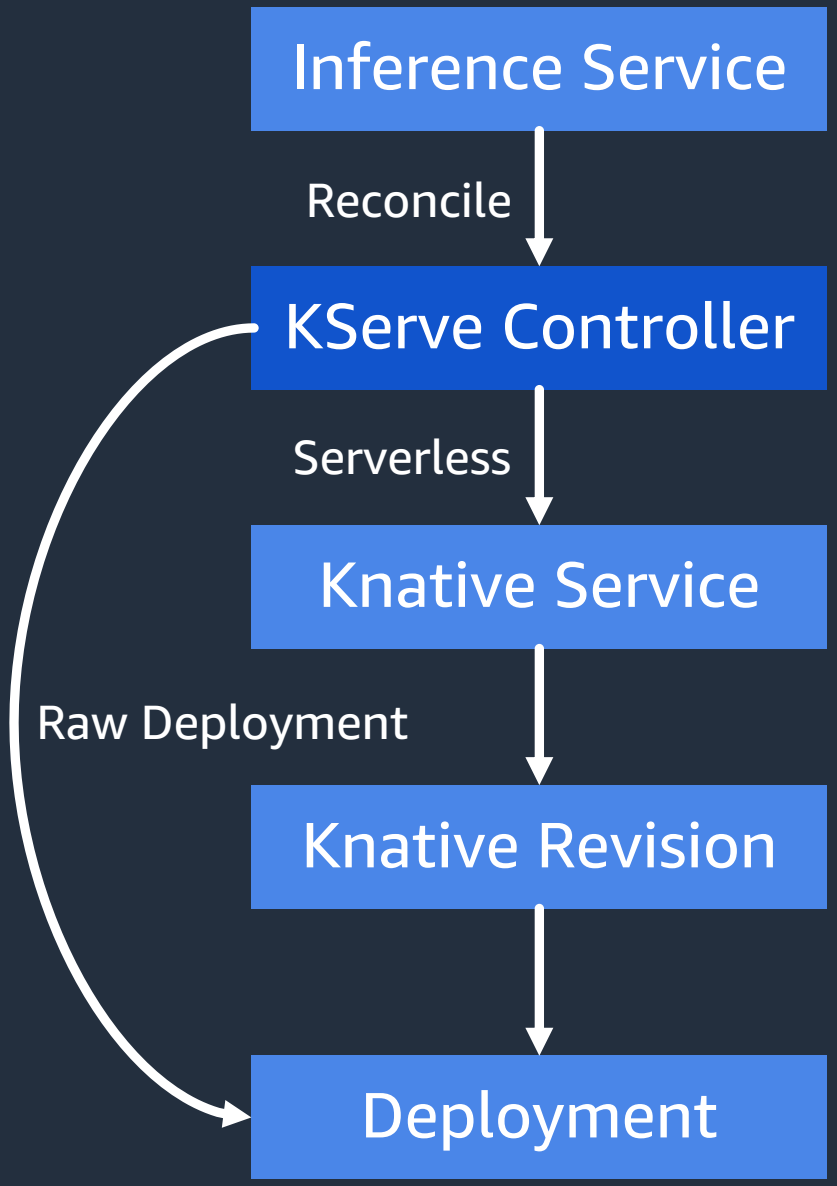




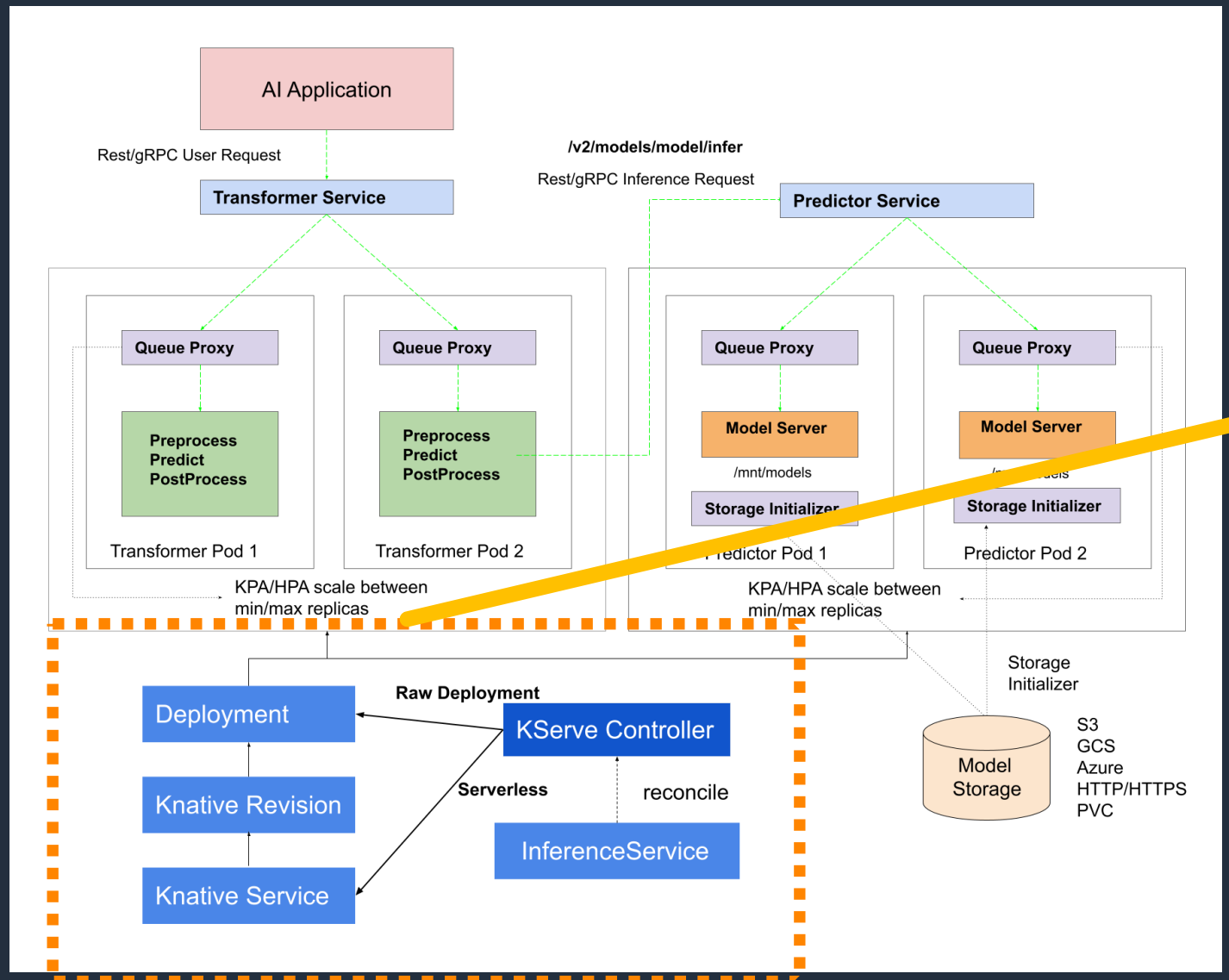
https://kserve.github.io/website/master/

# KServe Control Plane

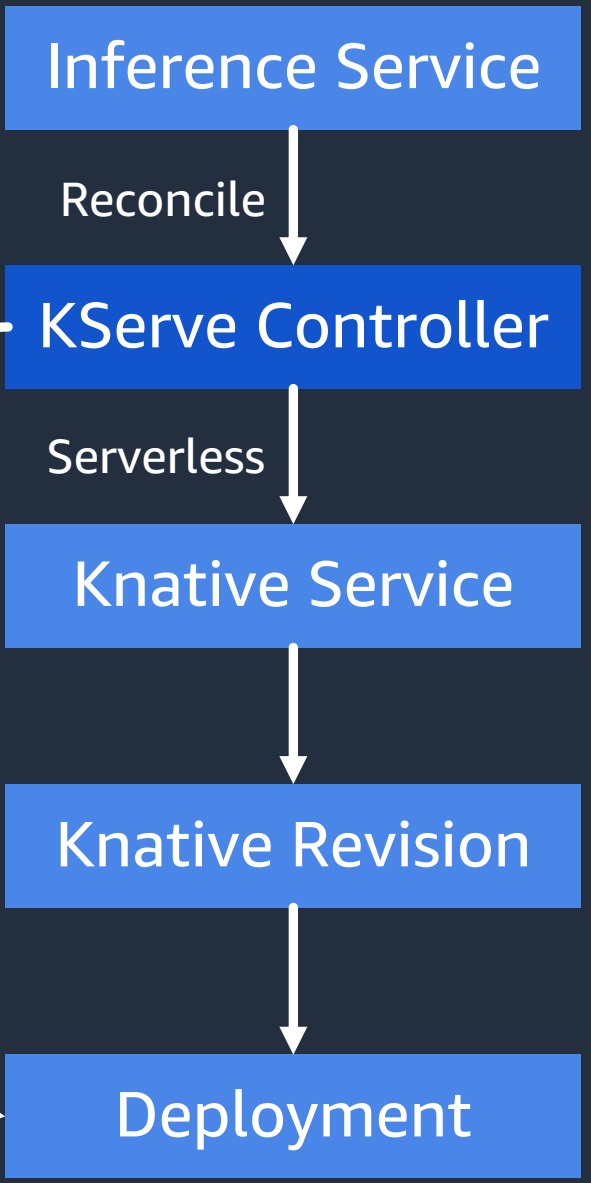


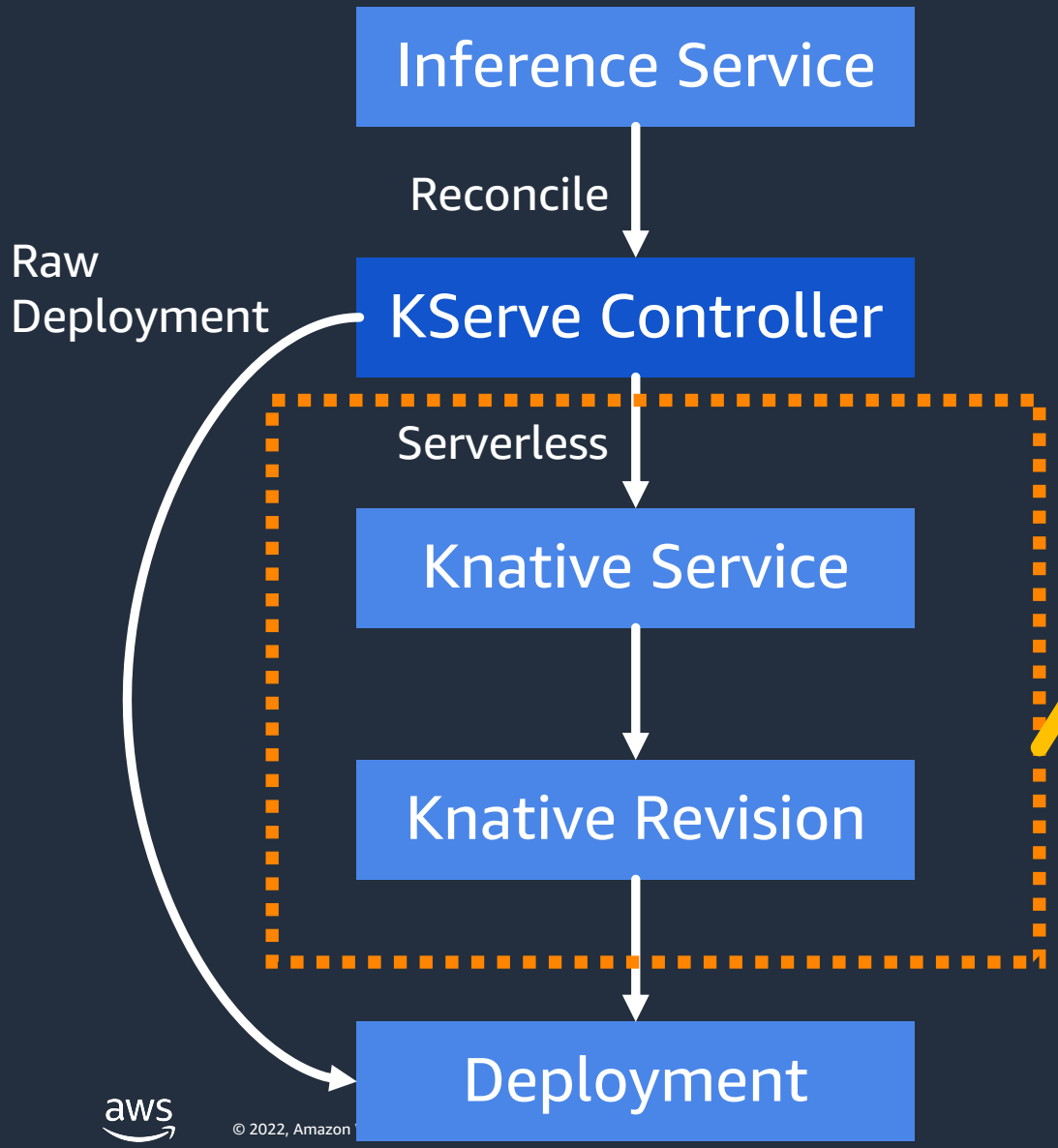
# KServe Control Plane

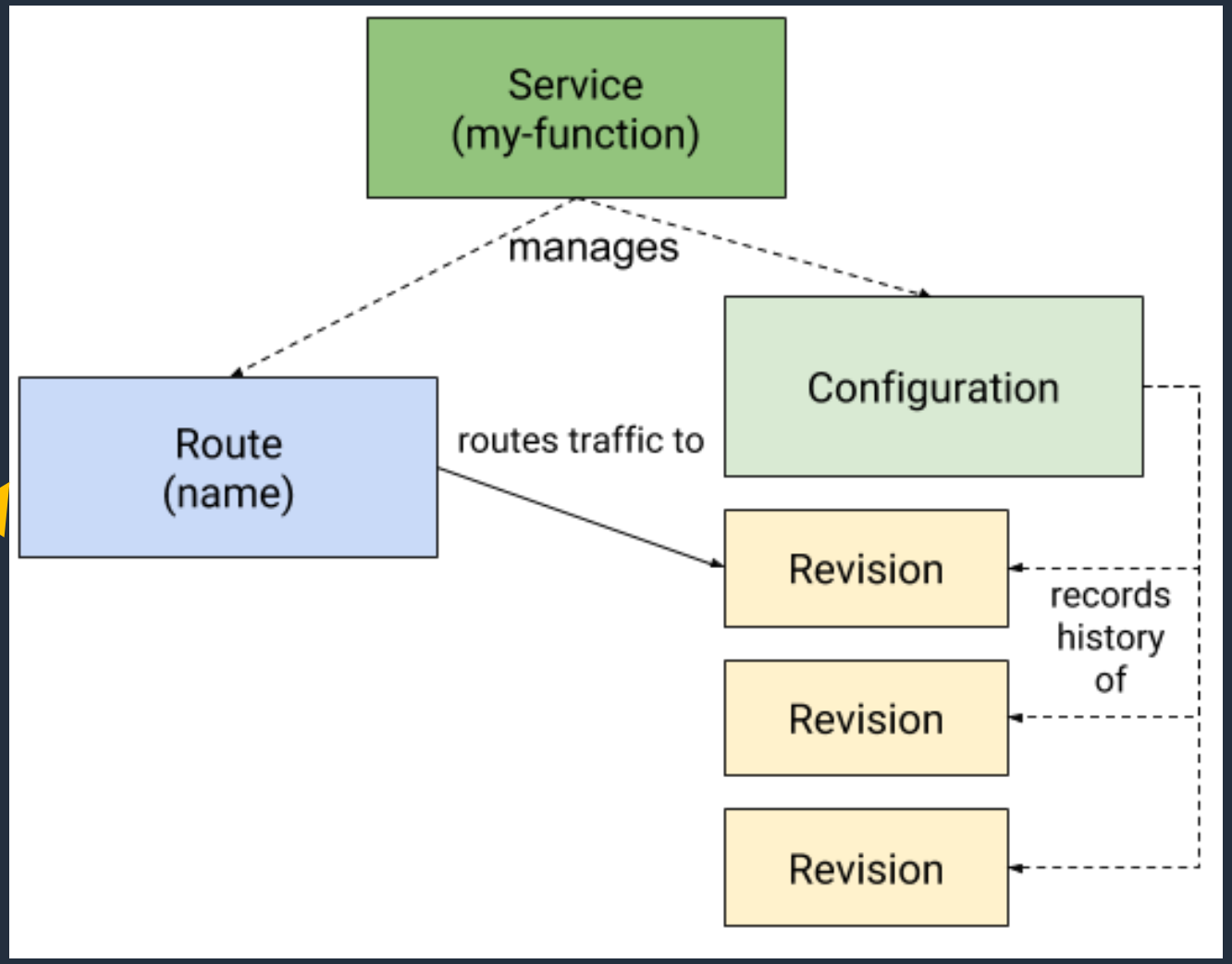



Raw Deployment



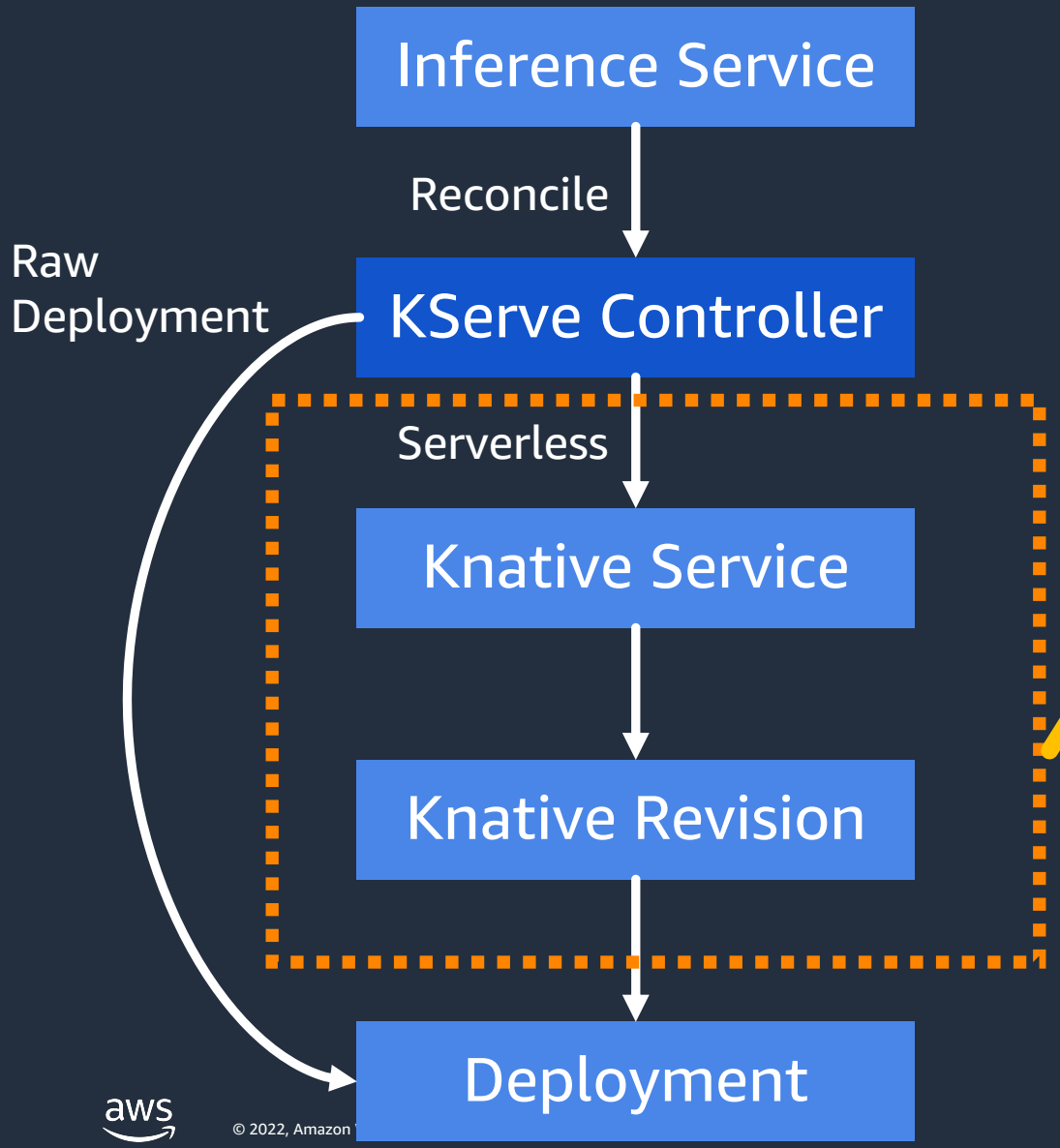
# Knative Components

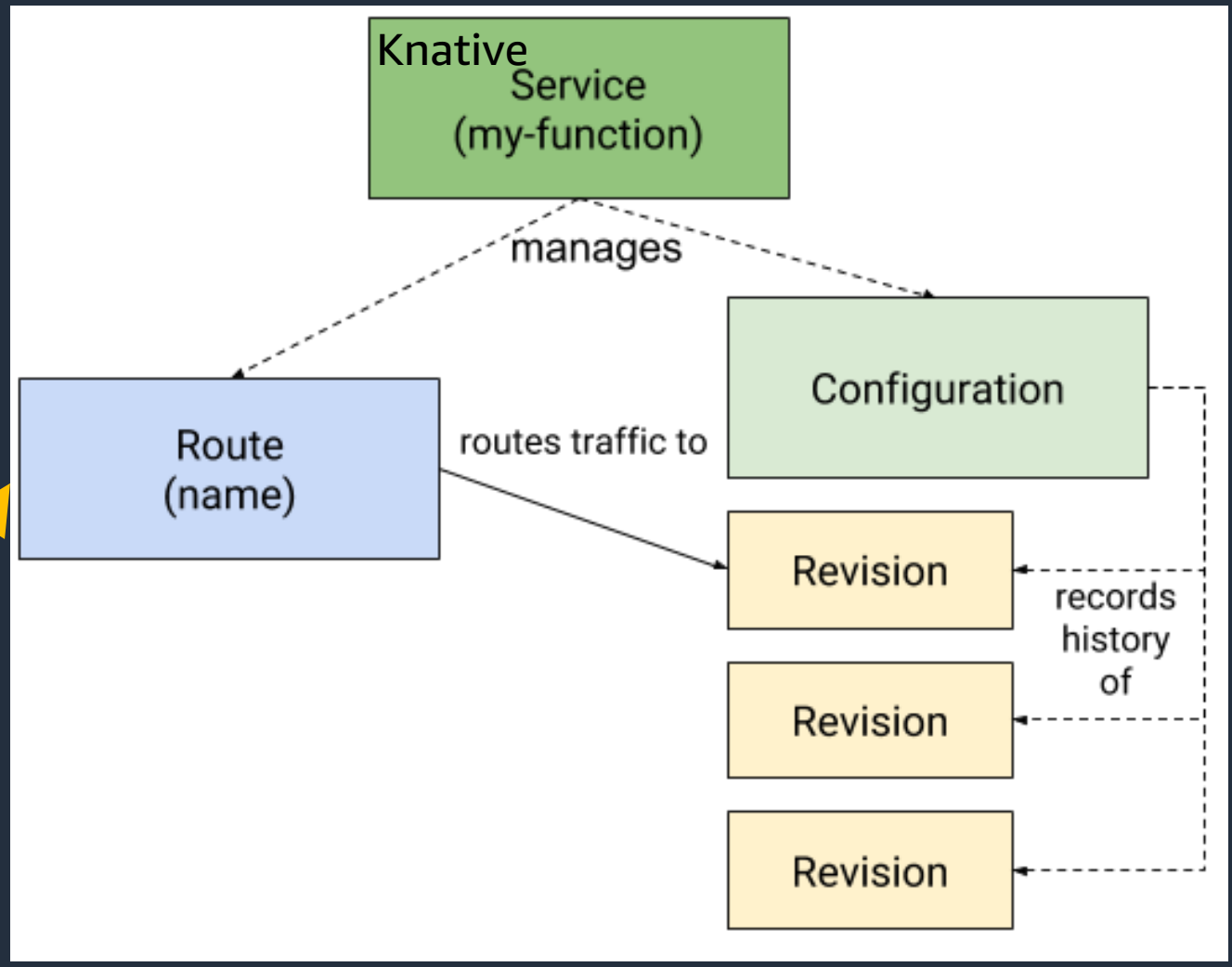



https://knative.dev/docs/serving/

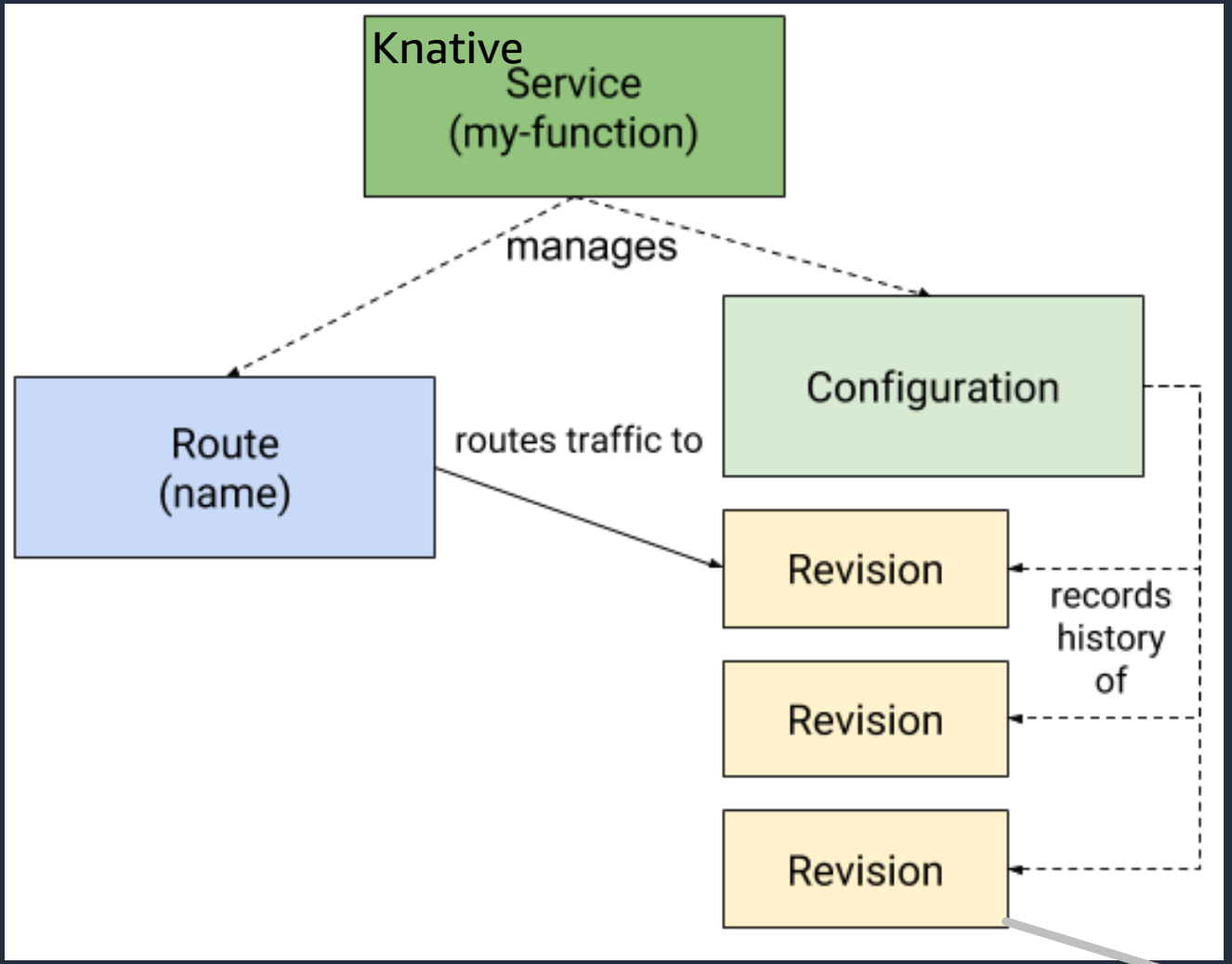
# Knative Serving




https://knative.dev/docs/serving/

# Primary Knative Serving Resources



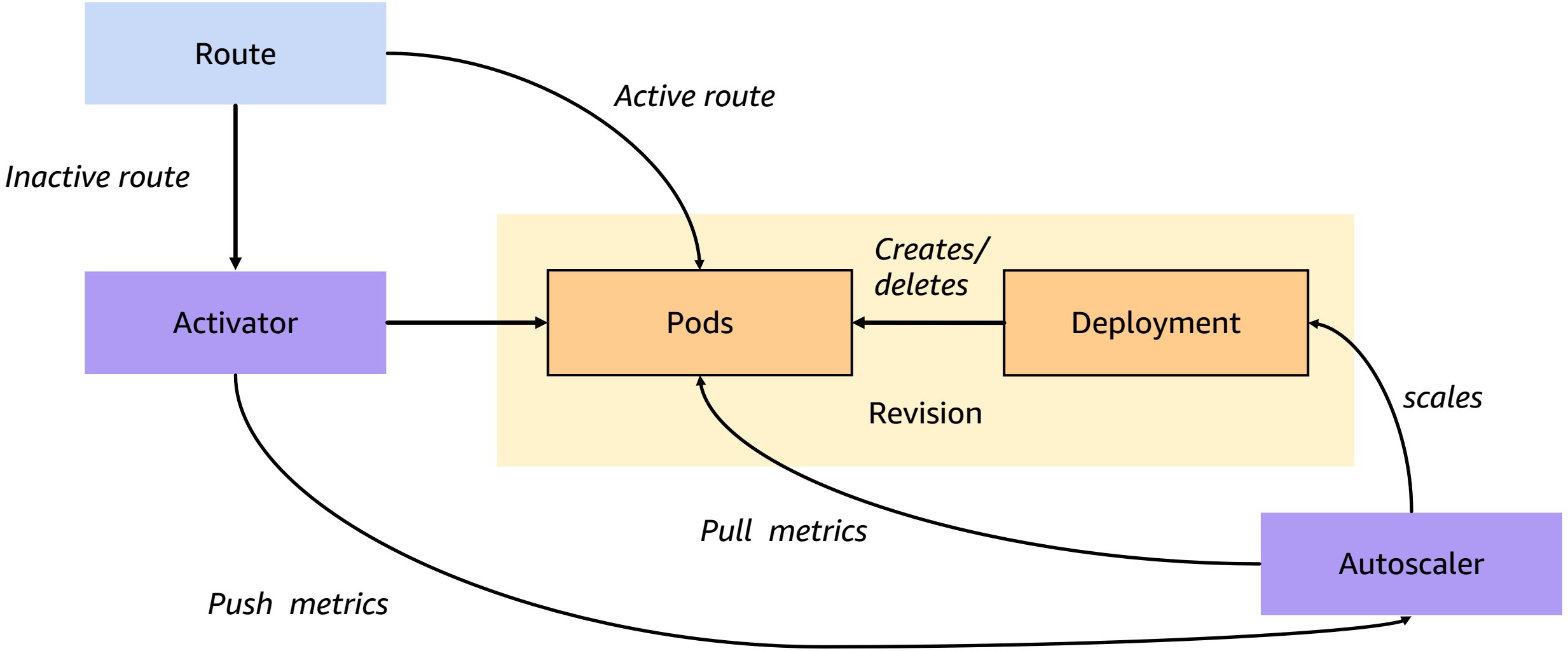

Knative Service resource automatically manages the whole lifecycle of your workload.

Routes maps a network endpoint to one or more revisions.

Configuration maintains the desired state for your deployment.

Revision is a point-in-time snapshot of the code and configuration for each modification made to the workload.

# Autoscaling with Knative Pod Autoscaler (KPA)

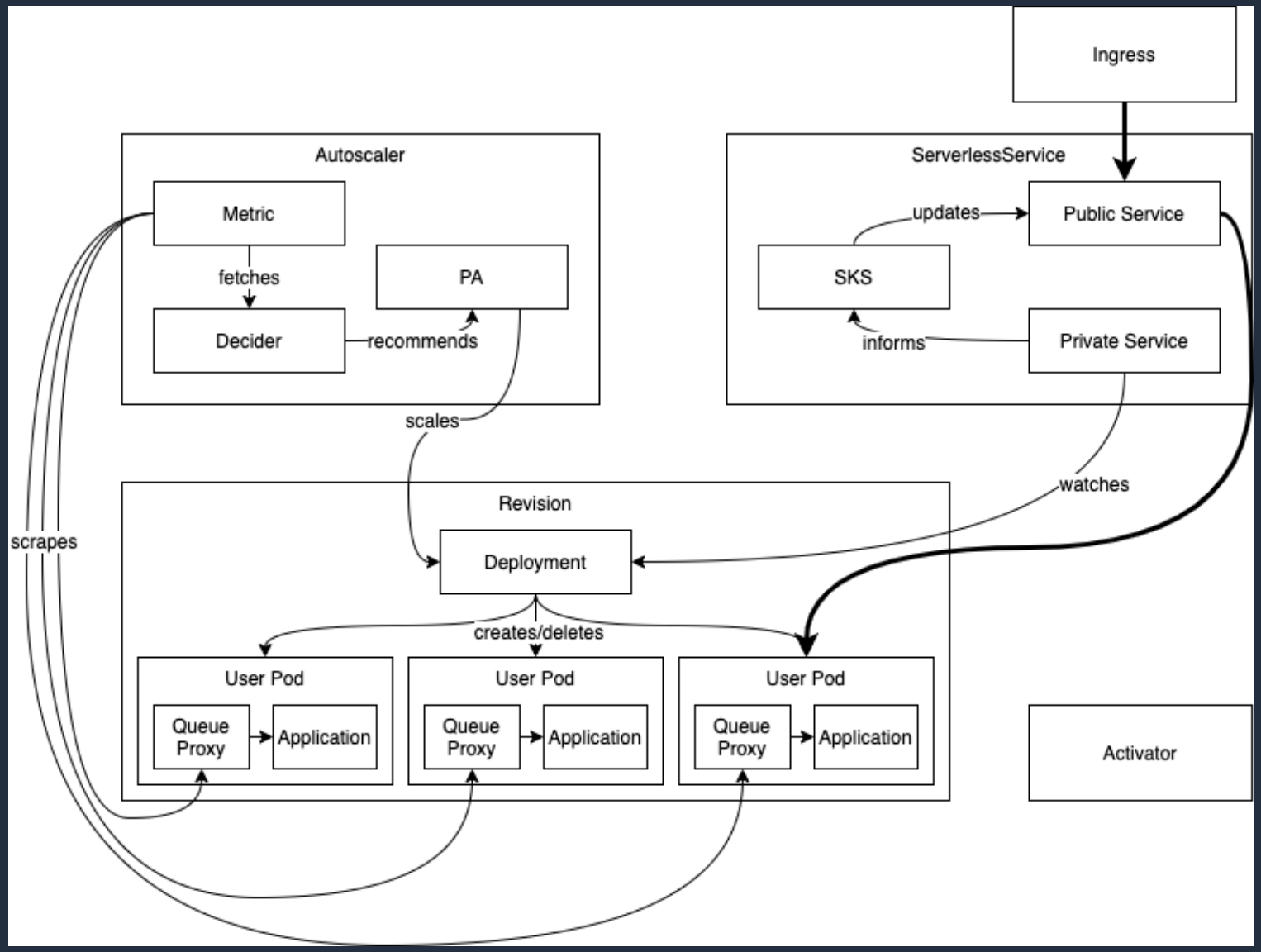







https://developer.aliyun.com/article/710828
https://knative.dev/docs/serving/istio-authorization/

# Scaling up and down (steady state)

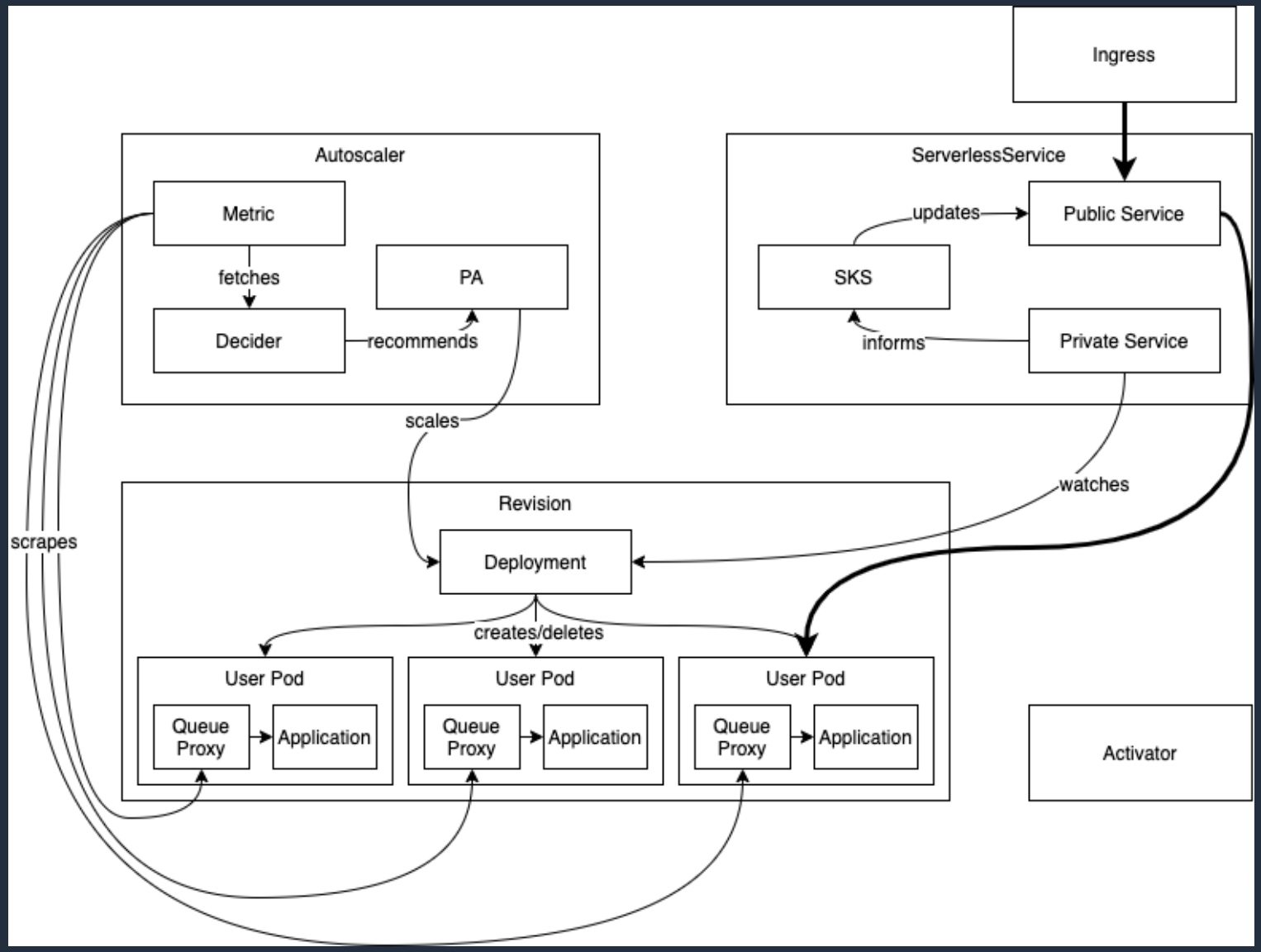







https://github.com/knative/serving/blob/main/docs/scaling/SYSTEM.md

# Scaling to zero

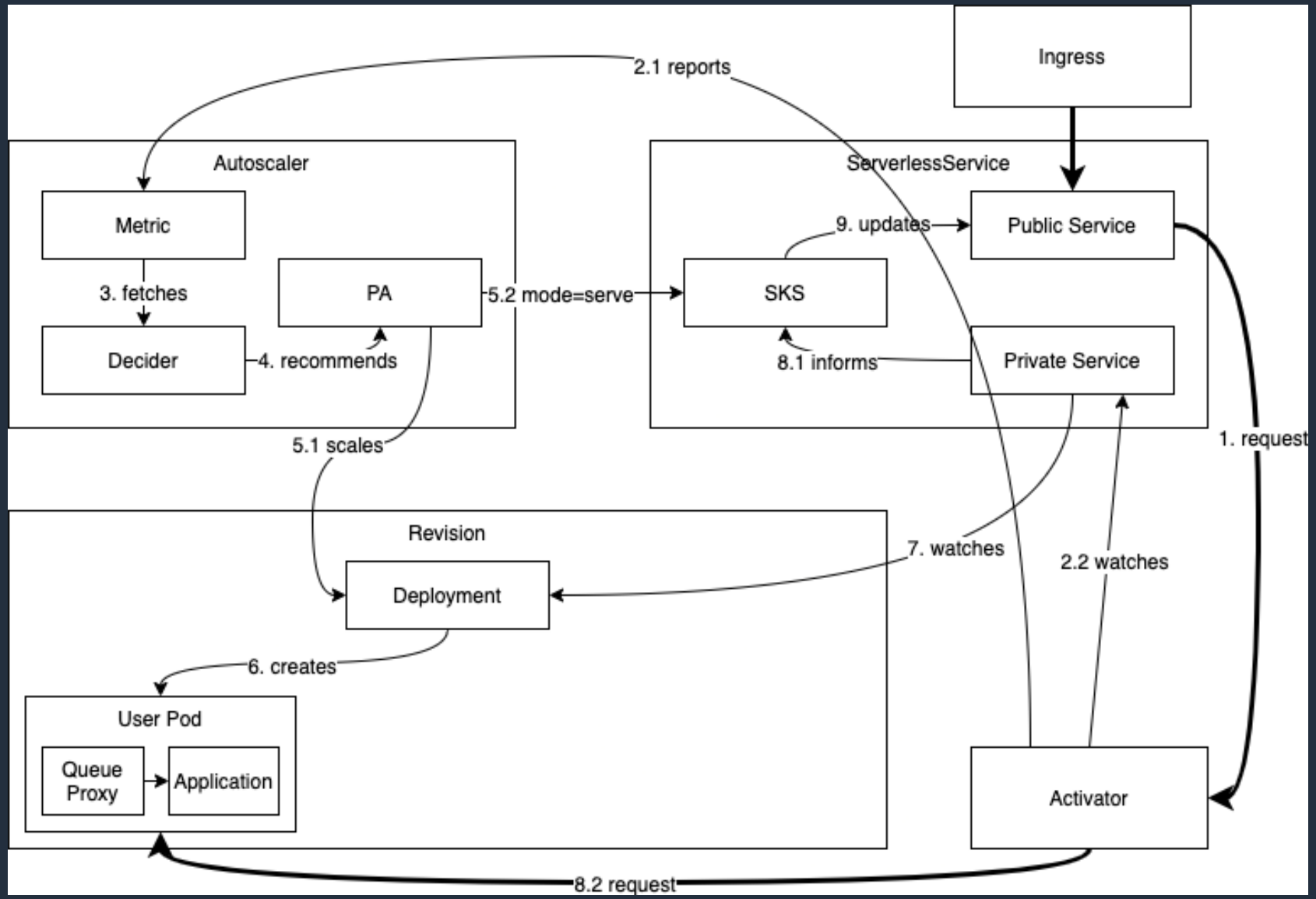







https://github.com/knative/serving/blob/main/docs/scaling/SYSTEM.md

# Scaling from Zero








https://github.com/knative/serving/blob/main/docs/scaling/SYSTEM.md

# Autoscale Sample

```yaml
apiVersion: serving.knative.dev/v1alpha1
kind: Service
metadata:
  name: autoscale-go
  namespace: default
spec:
  runLatest:
    configuration:
      revisionTemplate:
        spec:
          container:
            image: github.com/knative/docs/
            serving/samples/autoscale-go
```

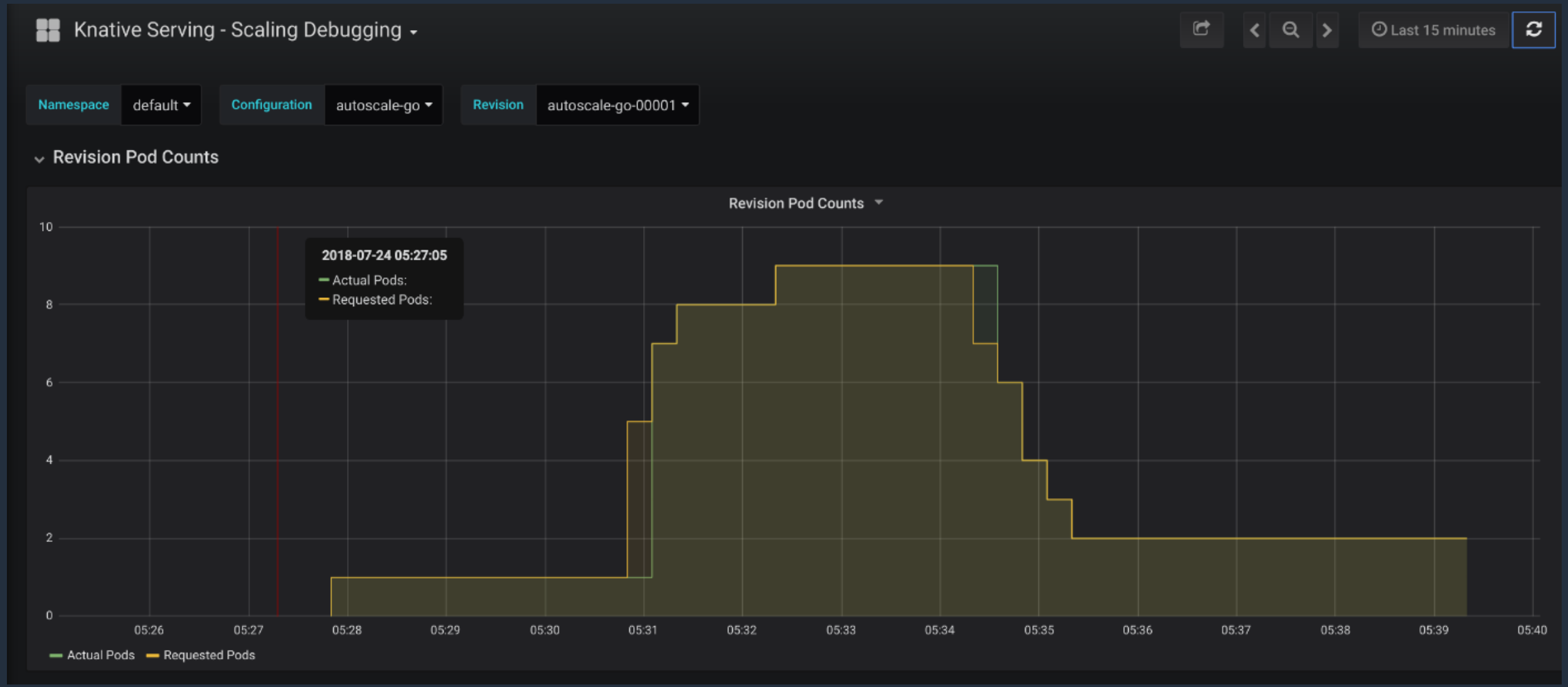
Ramp up traffic to maintain 10 in-flight requests.

```
go run serving/samples/autoscale-
go/test/test.go \
  -sleep 100 -prime 10000 -bloat 5 -qps 9999
-concurrency 300
```

```
REQUEST STATS:
Total: 439      Inflight: 299   Done: 439 ... Avg Latency: 0.4655 sec
Total: 1151     Inflight: 245   Done: 712 ... Avg Latency: 0.4178 sec
Total: 1706     Inflight: 300   Done: 555 ... Avg Latency: 0.4794 sec
Total: 2334     Inflight: 264   Done: 628 ... Avg Latency: 0.5207 sec
Total: 2911     Inflight: 300   Done: 577 ... Avg Latency: 0.4401 sec
...
```

https://github.com/dewitt/knative-docs/tree/master/serving/samples/autoscale-go

# Scaling pod from zero






https://github.com/dewitt/knative-docs/tree/master/serving/samples/autoscale-go

# Difference between KPA and HPA

## Knative Pod Autoscaler (KPA)

- Part of the Knative Serving core and enabled by default once Knative Serving is installed.
- Supports scale to zero functionality.
- Does not support CPU-based autoscaling.

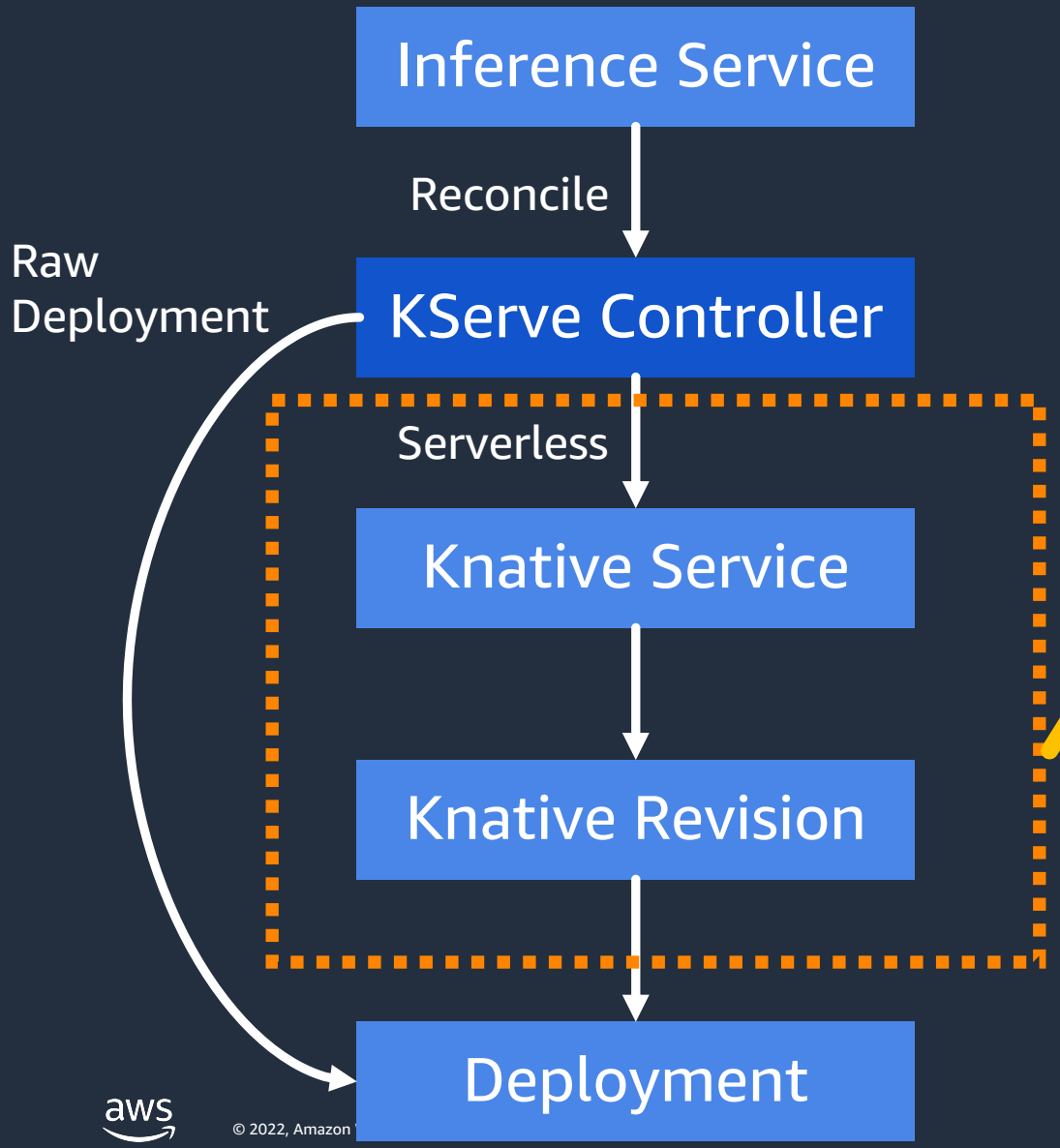
## Horizontal Pod Autoscaler (HPA)

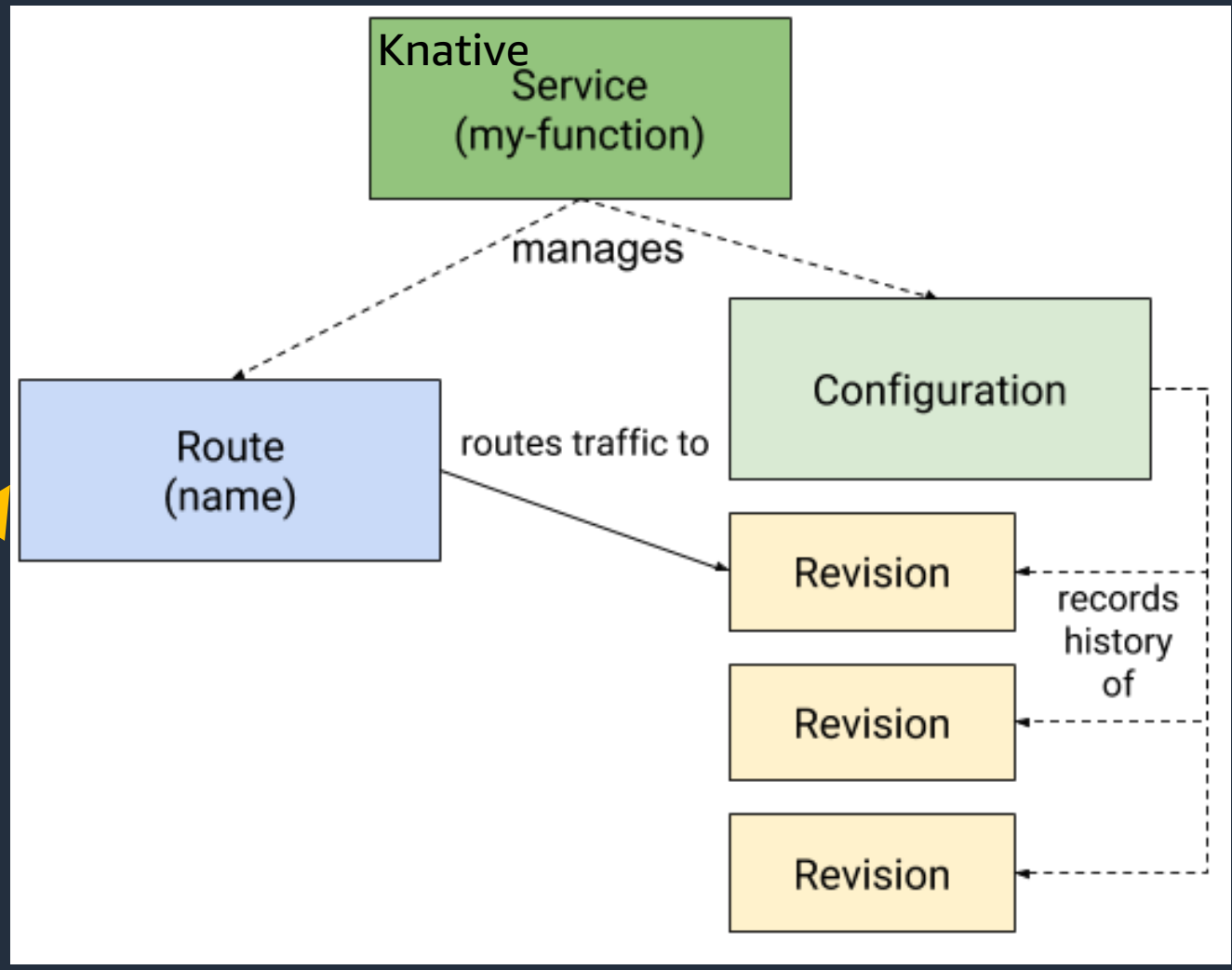
- Not part of the Knative Serving core, and must be enabled after Knative Serving installation.
- Does not support scale to zero functionality.
- Supports CPU-based autoscaling.



https://kserve.github.io/website/0.8/modelserving/v1beta1/torchserve/#autoscaling

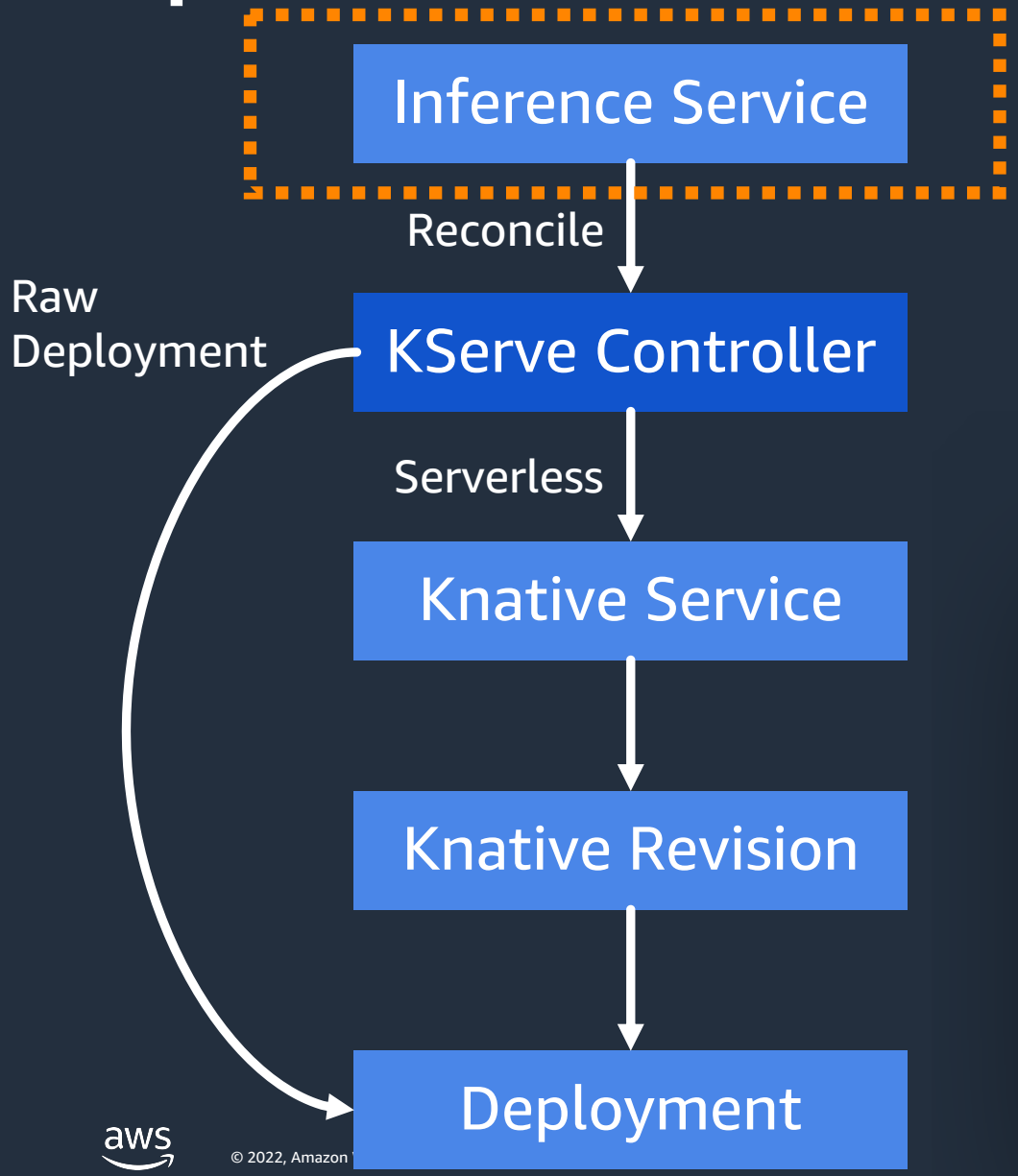
# We have covered Knative Serving part...




https://knative.dev/docs/serving/

# Up next: Inference Service




Question: Do we have to deal with the complexity in Knative?

Answer: No! All we need is Inference Service.

```yaml
apiVersion: "serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:
  name: "sklearn-iris"
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn
      storageUri: "gs://kfserving-
examples/models/sklearn/1.0/model"
```

# First InferenceSevice

```yaml
apiVersion:
"serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:
  name: "sklearn-iris"
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn
      storageUri: "gs://kfserving-
examples/models/sklearn/1.0/model"
```

Apply

# First Inference Service

```
cat <<EOF > "./iris-input.json"
{
  "instances": [
    [6.8,  2.8,  4.8,  1.4],
    [6.0,  3.4,  4.5,  1.6]
  ]
}
EOF
```

```
SERVICE_HOSTNAME=$(kubectl get inferenceservice
sklearn-iris -n kserve-test -o
jsonpath='{.status.url}' | cut -d "/" -f 3)

curl -v -H "Host: ${SERVICE_HOSTNAME}"
"http://${INGRESS_HOST}:${INGRESS_PORT}/v1/models
/sklearn-iris:predict" -d @./iris-input.jso
```

```
*   Trying 127.0.0.1:8081...
* Connected to localhost (127.0.0.1) port 8081
(#0)
> POST /v1/models/sklearn-iris:predict HTTP/1.1
> Host: sklearn-iris.kserve-test.example.com
...
<
* Connection #0 to host localhost left intact
{"predictions":[1,1]}
```

# First Inference Service: load test

```
# use kubectl create instead of apply because the job template
# is using generateName which doesn't work with kubectl apply
kubectl create -f https://raw.githubusercontent.com/
kserve/kserve/release-0.8/docs/samples/v1beta1/sklearn/v1/perf.yaml -n kserve-test
```

Under the hood

```
NAME                                                    READY   STATUS      RESTARTS   AGE
load-test95k5g-2qktp                                    0/1     Completed   0          102s
sklearn-iris-predictor-default-00001-deployment-77f7cbb54497gd8   2/2     Running     0          38m
```

```
Requests      [total, rate, throughput]         30000, 500.02, 499.55
Duration      [total, attack, wait]             1m0s, 59.998s, 56.662ms
Latencies     [min, mean, 50, 90, 95, 99, max]  3.581ms, 49.293ms, 49.148ms, 57.565ms, 62.676ms, 85.408ms, 153.4ms
Bytes In      [total, mean]                     630000, 21.00
Bytes Out     [total, mean]                     2460000, 82.00
Success       [ratio]                           100.00%
Status Codes  [code:count]                      200:30000
Error Set:
```

https://kserve.github.io/website/master/get_started/first_isvc/#5-perform-inference

# Thank you!